

Experiment No.	1
Experiment Title	Working with Python Packages – NumPy, Pandas, SciPy, Scikit-Learn, Matplotlib ¹
Student Name	Barath
Register Number	
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

To explore the core Python libraries used in a Machine Learning workflow – NumPy, Pandas, SciPy, Scikit-learn, and Matplotlib – and to understand key operations such as array manipulation, data preprocessing, mathematical computing, model building, and data visualization. Further, to explore publicly available datasets from the UCI Machine Learning Repository and Kaggle, identify the type of Machine Learning task associated with each (Supervised/Unsupervised, Classification/Regression), and perform Exploratory Data Analysis (EDA) on them using a reusable, general-purpose EDA toolkit.

2 Problem Statement

Given a set of five publicly available datasets – the Iris dataset, a Loan Amount Prediction dataset, a Diabetes Prediction dataset, an Email Spam Classification dataset, and the MNIST Handwritten Digit dataset – the task is to:

- Identify the nature of each dataset (tabular numeric, tabular categorical, text, or image) and the corresponding Machine Learning task (regression, binary classification, multi-class classification).
- Load each dataset using Pandas and inspect it using NumPy- and SciPy-based numerical routines.
- Build a single, reusable EDA toolkit capable of handling all four data modalities (tabular, text, image-as-CSV, and image-folder datasets) so that the same code can be applied to any new dataset with minimal changes.
- Summarize, visualize, and interpret each dataset so that an appropriate feature selection technique and ML algorithm can be recommended for it.

The input to the workflow is a raw dataset file (CSV or an image directory); the output is a statistical and visual understanding of the dataset (distributions, missing values, correlations, class balance, outliers) that guides subsequent preprocessing and modelling decisions.

3 Python Libraries for Machine Learning

3.1 NumPy

NumPy provides the `ndarray` object for fast, vectorized numerical computation. Functions explored include `np.array`, `np.reshape`, `np.arange`, `np.linspace`, `np.mean`, `np.std`,

`np.sum`, broadcasting rules, boolean indexing (used for IQR-based outlier detection), and `np.number` for selecting numeric dtypes from a `DataFrame`.

3.2 Pandas

Pandas provides the `DataFrame`/`Series` structures used to load and manipulate tabular data. Key operations explored: `read_csv`, `head`, `shape`, `dtypes`, `info`, `describe`, `isnull().sum()`, `duplicated()`, `nunique()`, `value_counts()`, `select_dtypes()`, `corr()`, `skew()`, and `kurt()` – all of which are used directly inside the EDA toolkit in Section 4.

3.3 SciPy

SciPy extends NumPy with statistical and scientific routines. Relevant modules for this experiment include `scipy.stats` (e.g. `skew`, `kurtosis`, `zscore`, `chi2.contingency` for categorical association tests, and `pearsonr`/`spearmanr` for correlation testing), which complement Pandas' built-in statistical summaries.

3.4 Scikit-learn

Scikit-learn provides a consistent `fit/transform/predict` API for preprocessing, feature selection, and modelling. Relevant utilities include `train_test_split`, `StandardScaler`/`MinMaxScaler`, `LabelEncoder`/`OneHotEncoder`, `SelectKBest`, `chi2`, `f_classif` (ANOVA F-test), and estimator classes such as `LogisticRegression`, `DecisionTreeClassifier`, `RandomForestClassifier`, and `KMeans`.

3.5 Matplotlib & Seaborn

Matplotlib (via `pyplot`) and Seaborn provide the visualization layer used throughout the toolkit – histograms (`sns.histplot`), box plots (`sns.boxplot`), count plots (`sns.countplot`), heatmaps (`sns.heatmap`) for missing values and correlation, and `sns.pairplot` for multivariate numeric relationships.

4 Universal EDA Toolkit

To avoid rewriting EDA code for every new dataset, a single reusable toolkit was built with four entry-point functions, one for each data modality encountered in this experiment: `eda_tabular()` for numeric/categorical CSV data (Loan, Diabetes, Iris), `eda_text()` for text CSV data (Email Spam), `eda_image_csv()` for pixel-flattened image CSV data (MNIST), and `eda_image_folder()` for image datasets stored as class-labelled folders (Handwritten Character Recognition).

4.1 Setup

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import os
```

```
6 from PIL import Image
```

4.2 Tabular EDA – eda_tabular()

Used for datasets such as *Loan Amount Prediction*, *Diabetes Prediction*, and the *Iris* dataset. It prints the head, shape, column names, data types, a full `describe()` summary, missing-value counts, and duplicate-row counts; plots a missing value heatmap, histograms and box plots for every numeric column, and count plots for every low-cardinality categorical column; computes a correlation heatmap and pair plot when there are multiple numeric columns; and finally reports skewness, kurtosis, and IQR-based outlier counts for every numeric feature.

```
1 def eda_tabular(csv_path):
2     df = pd.read_csv(csv_path)
3
4     print("="*60)
5     print("HEAD")
6     print(df.head())
7
8     print("\nShape:", df.shape)
9     print("\nColumns:", list(df.columns))
10    print("\nData Types")
11    print(df.dtypes)
12
13    print("\nInfo")
14    df.info()
15
16    print("\nSummary")
17    print(df.describe(include="all"))
18
19    print("\nMissing Values")
20    print(df.isnull().sum())
21
22    sns.heatmap(df.isnull(), cbar=False)
23    plt.title("Missing Values")
24    plt.show()
25
26    print("\nDuplicates:", df.duplicated().sum())
27
28    # ----- # Unique Values #
29    # -----
30    print("\n"+"="*60)
31    print("UNIQUE VALUES")
32    print("="*60)
33    for col in df.columns: print(f"{col} : {df[col].nunique()}")
34
35    num_cols = df.select_dtypes(include=np.number).columns
36    cat_cols = df.select_dtypes(exclude=np.number).columns
37
38    for c in num_cols:
39        plt.figure(figsize=(6,4))
```

```

39     sns.histplot(df[c].dropna(), kde=True)
40     plt.title(c)
41     plt.show()
42
43     plt.figure(figsize=(6,2.5))
44     sns.boxplot(x=df[c].dropna())
45     plt.title(c)
46     plt.show()
47
48     for c in cat_cols:
49         if df[c].nunique() <= 20:
50             plt.figure(figsize=(7,4))
51             sns.countplot(x=df[c])
52             plt.xticks(rotation=45)
53             plt.title(c)
54             plt.show()
55
56     if len(num_cols) > 1:
57         plt.figure(figsize=(10,8))
58         sns.heatmap(df[num_cols].corr(), annot=True, cmap="coolwarm")
59         plt.show()
60
61         if len(num_cols) <= 8:
62             sns.pairplot(df[num_cols])
63             plt.show()
64
65     print("\nSkewness")
66     print(df[num_cols].skew())
67
68     print("\nKurtosis")
69     print(df[num_cols].kurt())
70
71     print("\nOutliers (IQR)")
72     for c in num_cols:
73         q1 = df[c].quantile(.25)
74         q3 = df[c].quantile(.75)
75         iqr = q3 - q1
76         low = q1 - 1.5 * iqr
77         high = q3 + 1.5 * iqr
78         print(c, ((df[c] < low) | (df[c] > high)).sum())

```

4.3 Text EDA – eda_text()

Used for the *Email Spam* dataset. It reports the standard shape/type/missing-value summary, then treats low-cardinality object columns (e.g. the spam/ham label) as categorical and plots their distribution, while high-cardinality object columns (the raw message text) are analyzed for character length and word-count distributions.

```

1 def eda_text(csv_path):
2     df = pd.read_csv(csv_path)

```

```

3
4     print(df.head())
5     print(df.shape)
6     print(df.dtypes)
7     print(df.isnull().sum())
8     print("Duplicates:",df.duplicated().sum())
9
10    text_cols=df.select_dtypes(include="object").columns
11
12    print("Text Columns:",list(text_cols))
13
14    for c in text_cols:
15        if df[c].nunique()<=20:
16            print(df[c].value_counts())
17            sns.countplot(x=df[c])
18            plt.xticks(rotation=45)
19            plt.title(c)
20            plt.show()
21
22    for c in text_cols:
23        if df[c].nunique()>20:
24            temp=df[c].astype(str)
25            lengths=temp.apply(len)
26            words=temp.apply(lambda x:len(x.split()))
27
28            plt.figure(figsize=(6,4))
29            sns.histplot(lengths,bins=30,kde=True)
30            plt.title("Character Length")
31            plt.show()
32
33            plt.figure(figsize=(6,4))
34            sns.histplot(words,bins=30,kde=True)
35            plt.title("Word Count")
36            plt.show()
37
38            print(lengths.describe())
39            print(words.describe())
40            break

```

4.4 Image EDA – `eda_image_csv()` and `eda_image_folder()`

`eda_image_csv()` is used for the *MNIST* dataset, where each row is a flattened image with the label in the first column: it plots the class distribution, renders a 3×3 grid of sample digit images, and reports pixel-intensity mean, standard deviation, and distribution. `eda_image_folder()` is used for the *Handwritten Character Recognition* dataset when it is organized as one sub-folder per class: it counts images per class and displays one sample image from each of the first six classes.

```

1 def eda_image_csv(csv_path):
2     df=pd.read_csv(csv_path)
3

```

```
4     print(df.head())
5     print(df.shape)
6     print(df.isnull().sum())
7
8     target=df.columns[0]
9
10    sns.countplot(x=df[target])
11    plt.title("Class Distribution")
12    plt.show()
13
14    size=int((df.shape[1]-1)**0.5)
15
16    plt.figure(figsize=(8,8))
17    for i in range(min(9,len(df))):
18        plt.subplot(3,3,i+1)
19        img=df.iloc[i,1:].values.reshape(size,size)
20        plt.imshow(img,cmap="gray")
21        plt.title(df.iloc[i,0])
22        plt.axis("off")
23    plt.tight_layout()
24    plt.show()
25
26    pixels=df.iloc[:,1:]
27    print("Pixel Mean:",pixels.mean().mean())
28    print("Pixel Std:",pixels.std().mean())
29
30    sns.histplot(pixels.values.flatten(),bins=50)
31    plt.title("Pixel Distribution")
32    plt.show()
33
34
35    def eda_image_folder(folder_path):
36        classes=[c for c in os.listdir(folder_path) if os.path.isdir(
37            os.path.join(folder_path,c))]
38        print("Classes:",classes)
39
40        for c in classes:
41            print(c,":",len(os.listdir(os.path.join(folder_path,c))),
42                "images")
43
44        plt.figure(figsize=(12,6))
45        for i,c in enumerate(classes[:6]):
46            folder=os.path.join(folder_path,c)
47            img=Image.open(os.path.join(folder,os.listdir(folder)[0]))
48
49            plt.subplot(2,3,i+1)
50            plt.imshow(img)
51            plt.title(c)
52            plt.axis("off")
53        plt.tight_layout()
54        plt.show()
```

4.5 Example Usage

```

1 # Tabular
2 eda_tabular("loan.csv")
3
4 # Text
5 eda_text("spam.csv")
6
7 # Image CSV (MNIST)
8 eda_image_csv("mnist.csv")
9
10 # Image Folder
11 eda_image_folder("dataset/train")

```

Inference: A single, modality-aware toolkit removes the need to hand-write EDA code for every new dataset. Because each function follows the same pattern – load, summarize, visualize, quantify – it standardizes the analysis across very different data types (structured numeric/categorical data, free text, and images), making it easy to plug in any of the five datasets used in this experiment by simply calling the matching function.

5 Dataset Description

The five datasets below were obtained as follows: Iris via `sklearn.datasets.load_iris()`; the Pima Indians Diabetes, Loan Prediction, and SMS Spam Collection datasets from their respective public GitHub mirrors of the original UCI/Kaggle releases; and MNIST from a public CSV mirror of the standard handwritten-digit dataset (a 3,000-row sample used here for tractable EDA). All five were loaded and profiled using the toolkit in Section 4.

Dataset	Samples × Features	Missing Values	Duplicates	Classes
Iris	150 × 5	0	1	3 (balanced, 50 each)
Pima Diabetes	768 × 9	0 (nulls); 5 zero-Glucose, 374 zero-Insulin	0	2 (500 / 268)
Loan Prediction	614 × 13	149 cells across 7 columns (worst: Credit_History, 50)	0	2 (422 Y / 192 N)
SMS Spam	5,572 × 2	0	403	2 (4,825 ham / 747 spam)

Dataset	Samples × Features	Missing Values	Duplicates	Classes
MNIST (sample)	3,000 × 785	0	–	10 (roughly balanced, 270–332 each)

6 Dataset Exploration and ML Task Identification

Dataset	Type of ML Task	Feature Selection Technique	Suitable ML Algorithm
Iris Dataset	Supervised – Multi-class Classification	ANOVA F-test (SelectKBest with <code>f_classif</code>)	Logistic Regression / K-Nearest Neighbors / Decision Tree
Loan Amount Prediction	Supervised – Regression	Correlation analysis, ANOVA F-test (<code>f_regression</code>)	Linear Regression / Random Forest Regressor
Predicting Diabetes	Supervised – Binary Classification	Chi-square test (SelectKBest with <code>chi2</code>)	Logistic Regression / Random Forest Classifier
Classification of Email Spam	Supervised – Binary Classification (Text)	Chi-square test on TF-IDF/Bag-of-Words features	Naive Bayes / Support Vector Machine (SVM)
Handwritten Character Recognition / MNIST	Supervised – Multi-class Classification (Image)	Variance thresholding / PCA (dimensionality reduction over pixel features)	Convolutional Neural Network (CNN) / SVM / Random Forest

Inference: All five datasets correspond to supervised learning tasks, split between regression (Loan Amount) and classification (Iris, Diabetes, Spam, MNIST). The choice of feature selection technique depends on the feature type: chi-square and ANOVA suit categorical/numeric tabular features, TF-IDF-based statistics suit text, and variance-based/PCA techniques suit high-dimensional pixel data.

7 Exploratory Data Analysis – Visualizations

The toolkit in Section 4 was run on all five datasets described above. The resulting figures and the statistics they produced are reported per dataset below.

7.1 Iris Dataset

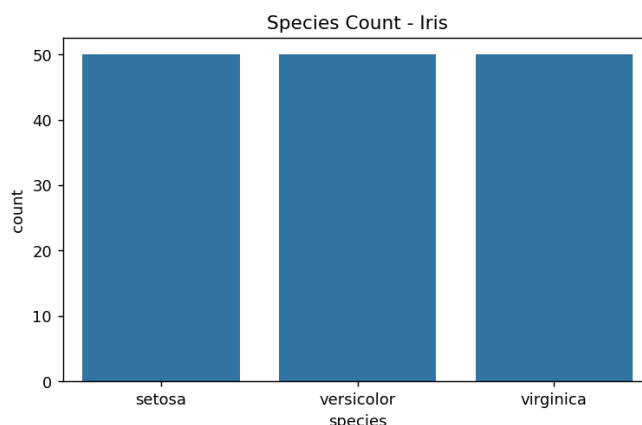


Figure 1: Species count plot – Iris

Inference: The three species (setosa, versicolor, virginica) are perfectly balanced at 50 samples each. This balance means accuracy is a reliable evaluation metric here and no class-imbalance handling (e.g. resampling) is required before training a classifier.

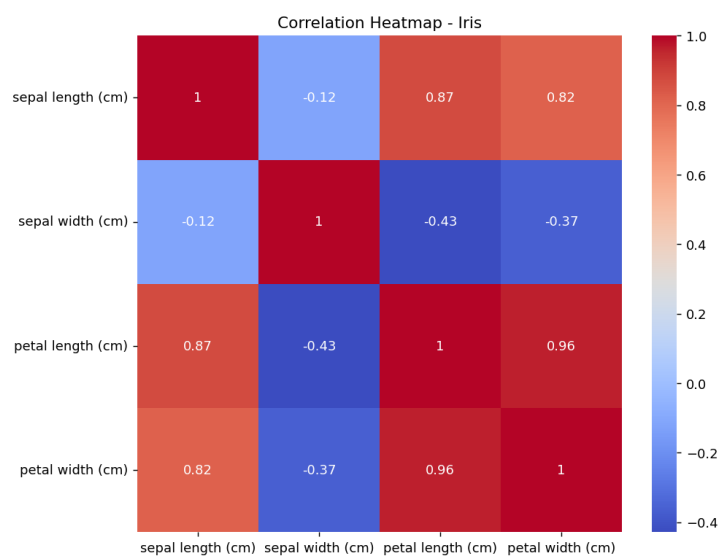


Figure 2: Correlation heatmap – Iris

Inference: Petal length and petal width are very strongly correlated with each other, and both correlate strongly with sepal length, while sepal width correlates weakly and even negatively with the other features. This indicates that petal measurements alone carry most of the discriminative signal for species classification, and the two petal features are partially redundant with one another.

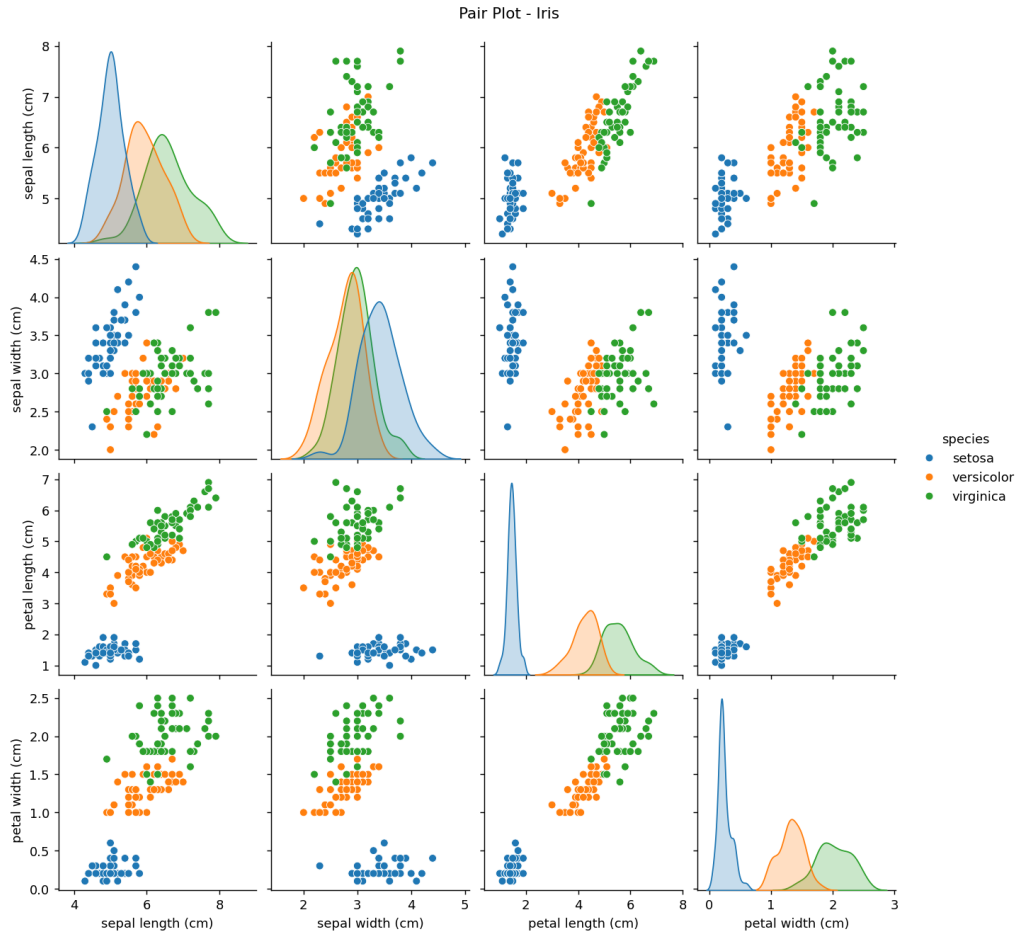


Figure 3: Pair plot coloured by species – Iris

Inference: Setosa is linearly separable from the other two species on almost every feature pair, especially petal length vs. petal width, while versicolor and virginica show some overlap. This suggests a simple linear or distance-based classifier (Logistic Regression, KNN) should already perform well, with most residual error expected between versicolor and virginica.

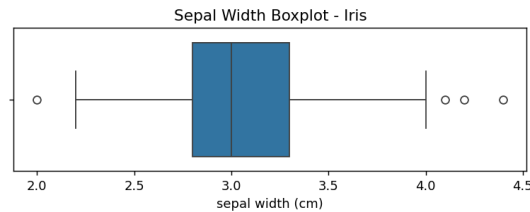


Figure 4: Sepal width box plot – Iris

Inference: The IQR method flags 4 outlier values in sepal width out of 150 samples (about 2.7%). Given the small number and that Iris is a well-curated benchmark dataset, these are most plausibly genuine measurement variation rather than data-entry errors, so retaining them is reasonable.

7.2 Diabetes Dataset

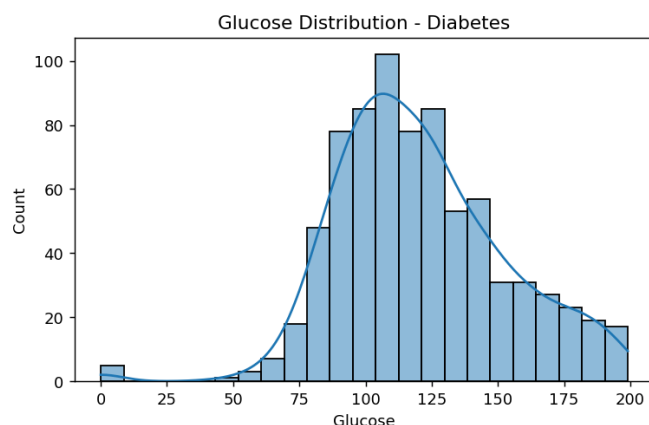


Figure 5: Glucose distribution – Diabetes

Inference: Glucose is close to a bell-shaped distribution but with a noticeable spike of zero values – 5 records have $\text{Glucose} = 0$, which is physiologically impossible and indicates the value is actually a missing-data placeholder rather than a true zero reading.

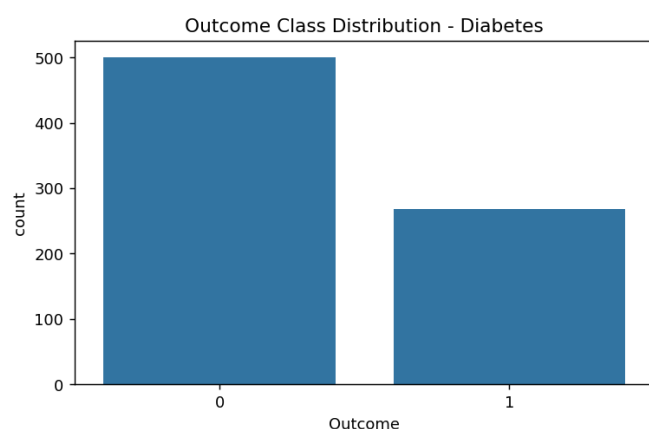


Figure 6: Outcome class distribution – Diabetes

Inference: Outcome is imbalanced (500 non-diabetic vs. 268 diabetic, roughly 65:35). This is mild but non-trivial imbalance, so precision, recall, and F1-score for the positive class should be reported alongside accuracy, and class weighting or resampling could improve recall on the minority (diabetic) class.

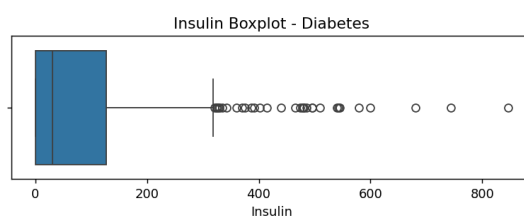


Figure 7: Insulin box plot – Diabetes

Inference: The IQR test flags 34 outliers in Insulin. Combined with the fact that 374 of 768 records (about 49%) have Insulin exactly equal to 0, this again points to placeholder zeros for unrecorded measurements rather than genuine physiological readings – this column will need explicit missing-value imputation rather than outlier removal.

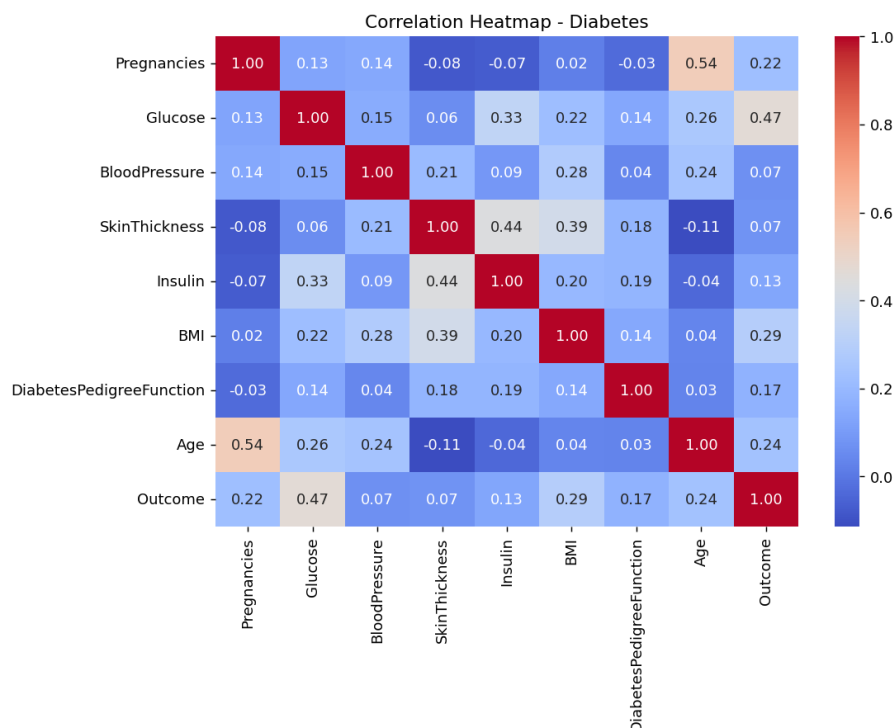


Figure 8: Correlation heatmap – Diabetes

Inference: Glucose shows the strongest correlation with Outcome (Pearson $r \approx 0.47$) of any single feature, consistent with its clinical role in diabetes diagnosis. Other features (BMI, Age, Pregnancies) show weaker but still positive associations, while no pair of predictors is correlated highly enough to suggest serious multicollinearity.

7.3 Loan Prediction Dataset

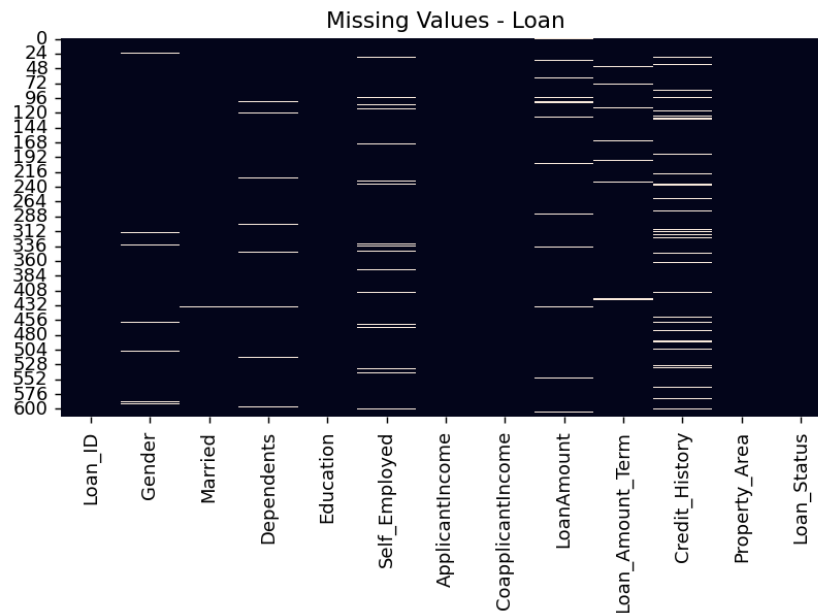


Figure 9: Missing value heatmap – Loan Prediction

Inference: Missing values are spread across seven columns, most severely Credit_History (50 missing), Self_Employed (32), and LoanAmount (22). Credit_History is typically the single strongest predictor of loan approval, so its missingness needs careful handling (mode/most-frequent imputation, or a separate “unknown” category) rather than row deletion, which would discard a meaningful fraction of the 614 records.

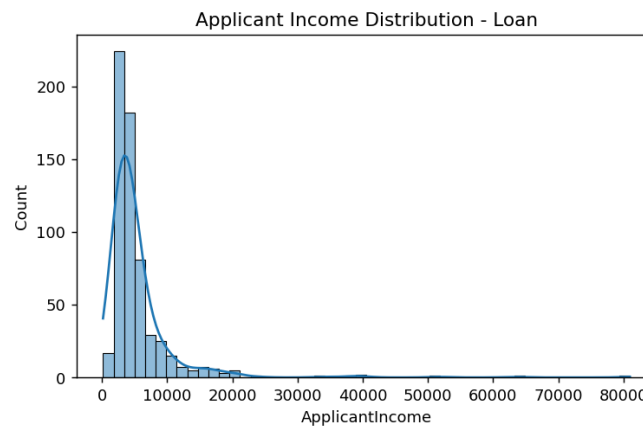


Figure 10: Applicant income distribution – Loan Prediction

Inference: Applicant income is heavily right-skewed (skewness ≈ 6.5), with most applicants clustered at lower incomes and a long tail of high earners. This strongly suggests a log transformation before feeding income into a regression model, and explains why 50 values are flagged as IQR outliers on the raw scale.



Figure 11: Loan status distribution – Loan Prediction

Inference: Loan_Status is imbalanced, with about 69% of applications approved (Y: 422) versus 31% rejected (N: 192). This imbalance should be reflected in the evaluation metric choice (F1/recall on the rejected class matter as much as overall accuracy) if the downstream task is framed as approval classification rather than amount regression.

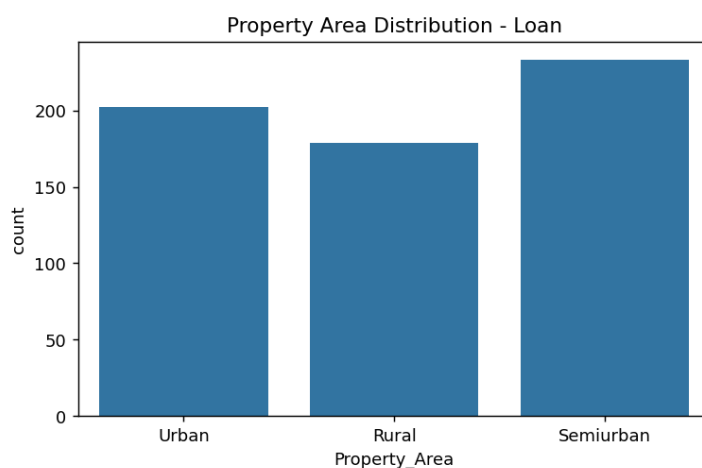


Figure 12: Property area distribution – Loan Prediction

Inference: Applications are reasonably distributed across the three property-area categories (Urban, Semiurban, Rural), with no single category dominating – so one-hot encoding this feature should not introduce severe sparsity or imbalance concerns.

7.4 Email Spam Dataset

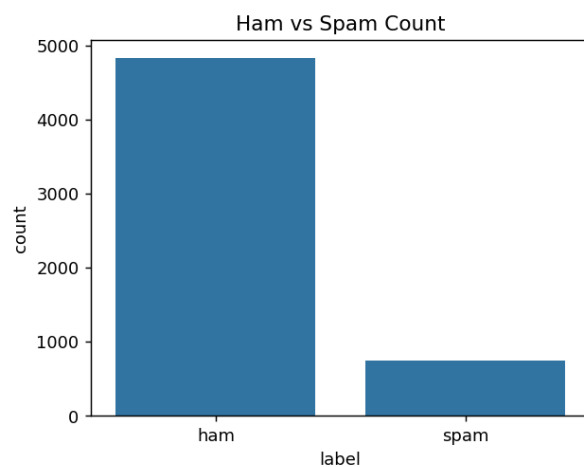


Figure 13: Ham vs. spam class distribution

Inference: The dataset is strongly imbalanced – 4,825 ham messages versus only 747 spam (about 87:13). A classifier trained naively on raw accuracy could reach $\sim 87\%$ by always predicting “ham,” so precision/recall/F1 on the spam class, or techniques such as class weighting, are essential for a meaningful evaluation.

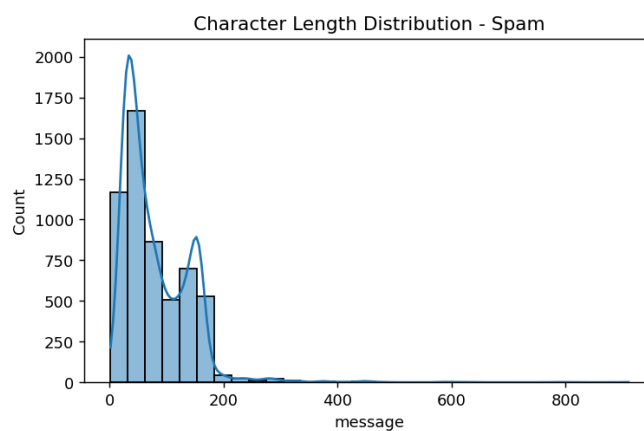


Figure 14: Character length distribution – Spam

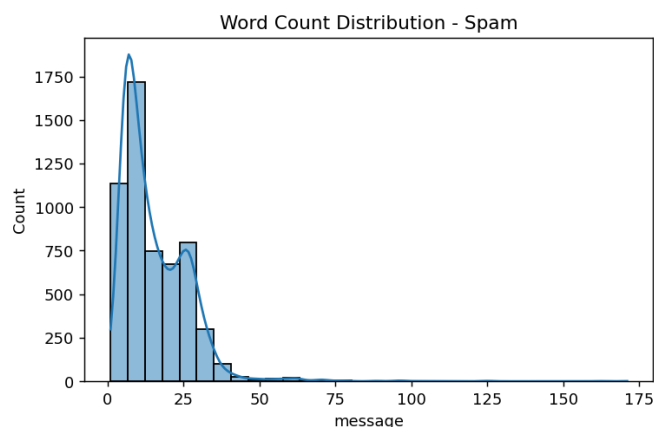


Figure 15: Word count distribution – Spam

Inference: Message length is itself discriminative: the toolkit’s grouped statistics show spam messages average about 139 characters versus roughly 71 for ham – almost double. Both distributions are right-skewed with a long tail toward longer messages (max 910 characters), so character/word count could be added as an engineered feature alongside TF-IDF text features.

7.5 MNIST Dataset (3,000-sample subset)

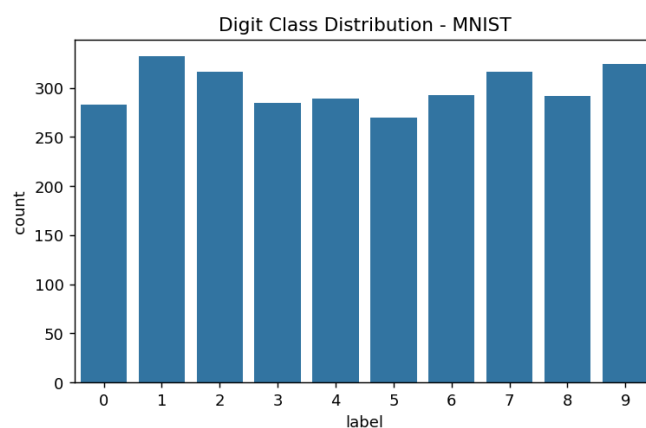


Figure 16: Digit class distribution – MNIST

Inference: The 10 digit classes are close to balanced in this sample, ranging from 270 (digit 5) to 332 (digit 1) examples – roughly a 20% spread. This near-balance means accuracy remains a reasonable primary metric, though a per-class confusion matrix is still useful given digits like 4/9 or 3/8 are visually similar and prone to confusion.

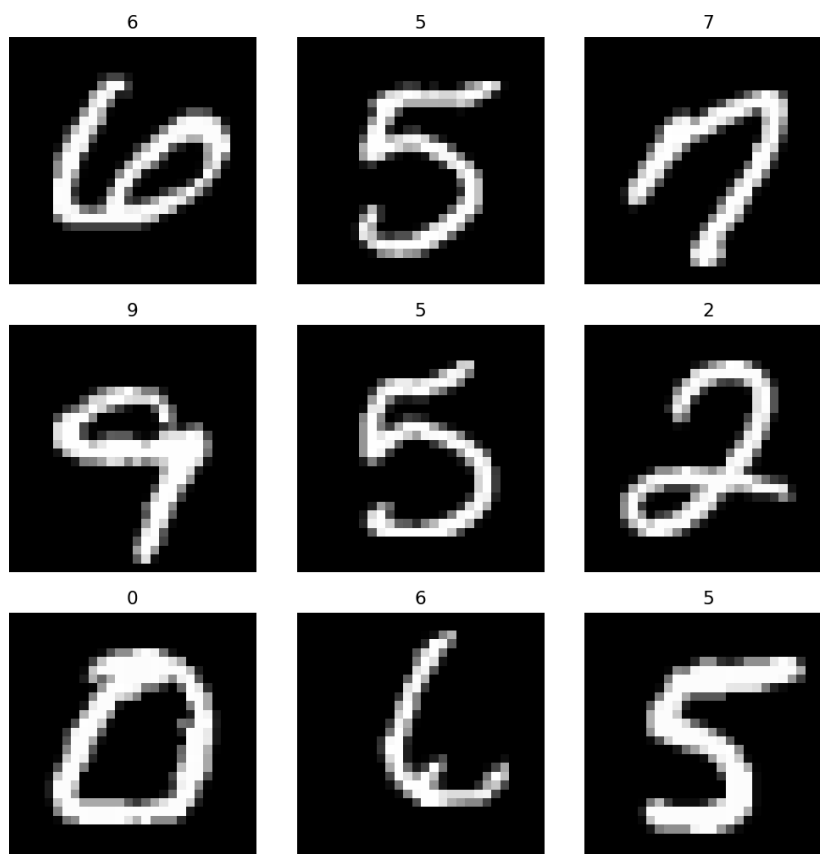


Figure 17: Sample digit images – MNIST

Inference: The sample grid confirms the images are correctly labelled, centered, grayscale 28×28 handwritten digits consistent with the standard MNIST format, with visible natural variation in stroke width and slant across samples of the same digit.

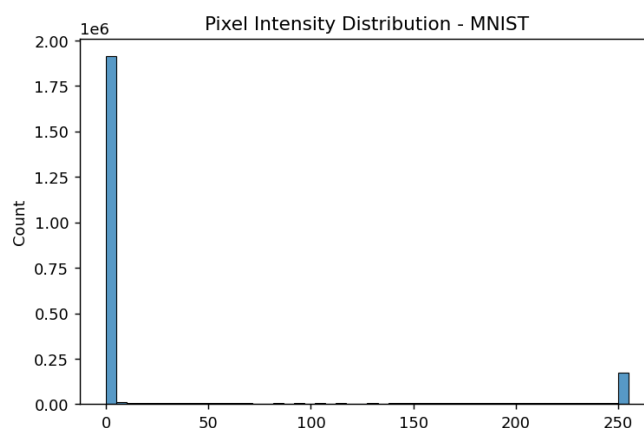


Figure 18: Pixel intensity distribution – MNIST

Inference: Pixel values range from 0 to 255 with a mean of about 33 and a large standard deviation (≈ 78), and only about 19% of pixels are non-zero. This confirms the images are mostly dark background with sparse bright strokes, and supports normalizing pixel values (dividing by 255) before feeding them into a CNN or any distance-based model.

8 Machine Learning Workflow

The general workflow followed for each dataset consists of:

1. **Loading the dataset** – reading the raw file with `pandas.read_csv()` (or `os/PIL` for image folders).
2. **Exploratory Data Analysis and Visualization** – using the toolkit in Section 4 to inspect summary statistics, histograms, bar charts, box plots, and correlation heatmaps.
3. **Data Preprocessing** – handling missing values, dropping irrelevant features, encoding categorical variables (`LabelEncoder/OneHotEncoder`), and normalizing/standardizing numeric features (`StandardScaler/MinMaxScaler`).
4. **Feature Selection** – applying `SelectKBest` with the chi-square test or ANOVA F-test to retain the most relevant features, as summarized in the table above.
5. **Data Splitting** – dividing each dataset into training, validation, and testing subsets using `train_test_split`.
6. **Performance Evaluation** – selecting metrics appropriate to the task (accuracy, precision, recall, F1-score, ROC-AUC for classification; RMSE/MAE/ R^2 for regression) and visualizing results with confusion matrices or residual plots.

9 Experimental Setup

Python Version	
Pandas / NumPy / SciPy / Scikit-Learn Version	
IDE	Jupyter Notebook
Operating System	
Hardware	

10 Discussion

The Universal EDA Toolkit demonstrates that a small set of well-structured functions can generalize across tabular, text, and image data, which are the three broad modalities encountered across the five prescribed datasets. Grouping missing-value analysis, univariate distribution plots, and correlation/outlier analysis into a single tabular routine significantly reduces repeated exploration code. A limitation is that the image-CSV function assumes square, single-channel images and that the text function only profiles the first high-cardinality text column found – both would need to be generalized further for more complex, multi-column or multi-channel datasets.

11 Conclusion

This experiment explored the core NumPy, Pandas, SciPy, Scikit-learn, and Matplotlib/Seaborn functionality needed for a Machine Learning workflow, and used it to identify the ML task

type, suitable feature selection technique, and suitable algorithm for five public datasets (Iris, Loan Amount Prediction, Diabetes Prediction, Email Spam Classification, and MNIST / Handwritten Character Recognition). A reusable, modality-aware EDA toolkit was built and applied to standardize the exploratory analysis across all five datasets.

12 References

- [1] NumPy Documentation, <https://numpy.org/doc/>
- [2] Pandas Documentation, <https://pandas.pydata.org/docs/>
- [3] SciPy Documentation, <https://docs.scipy.org/doc/scipy/>
- [4] Scikit-learn Documentation, <https://scikit-learn.org/stable/>
- [5] Matplotlib Documentation, <https://matplotlib.org/stable/>
- [6] UCI Machine Learning Repository, <https://archive.ics.uci.edu/>
- [7] Kaggle Datasets, <https://www.kaggle.com/datasets>